

Lab Session Week 19

Today's topics

1. Correction to Lecture 9
 2. Relevant course content
 3. Hints for preparation
 4. Additional exercises
-

Correction to Lecture 9

In the part on *Open Addressing* in Lecture 9, I suggest a way to handle deletion that *does not work*. See the example solution to Assignment 3, Task 4.1 for a detailed discussion.

Relevant course content

- Below is an overview over the course content as taught.
- I added some comments on important aspects. Those comments are **not** a complete enumeration of everything relevant for the exam. In particular, I did not mention all aspects for all items of the curriculum to avoid repetition.
- Style of questions will be similar to those in the 2025 exam.

Course content

1. Introduction
 - Analyzing algorithms
 - Designing algorithms
2. Growth of functions
 - O , Ω , Θ
 - Proving Θ by finding constants
3. Proving the correctness of algorithms
 - Invariants and proofs for invariants
4. Divide & Conquer
 - Maximum-subarray problem
 - Recursion trees
 - Solving recurrences with the master method
5. Randomized algorithms

- Indicator variables
- Analysis of randomized algorithms

6. Sorting

- How do different sort algorithms work?
- Performance for different type of input data
- Simple sort algorithms, e.g., insertion or selection sort
- More efficient algorithms, e.g., merge sort, quicksort, heapsort
- Counting sort
- Priority queues

7. Data structures

- How are the data structures defined?
- What are their important properties?
- Which algorithms are defined for them and how do they work?
- Time and memory requirements?
- Applications?
- Structures studied
 - Heaps
 - Stacks & queues
 - Linked lists
 - Hash tables

8. Trees

- Definition, properties, algorithms, applications
- Basics of trees
- Binary search trees

9. Some algorithms

- Dynamic programming / Optimal substructure
- Greedy choice
- Similarities and differences between dynamic programming and greedy choice
- Examples of algorithms and applications
- Concrete algorithms
 - Rod cutting/Dynamic programming
 - Greedy algorithms

10. Graphs

- Defining properties, types of graphs
- Representations of graphs
- Graph algorithms
 - Elementary graph algorithms (eg traversal)
 - Breadth-first search
 - Depth-first search

11. Limits of computability (there will be no exam questions on this)

Additional exercises

- Try to do all of these exercises using only the resources you will have during the exam: your head and hands, paper, pen, pocket calculator and Cormen's *Algorithms unlocked* with your handwritten notes.
- Work and discuss with fellow students, try to explain solutions to each other.
- Feel free to use AI agents to generate more training material, but carefully check answers provided.
- Review previous exams posted on Canvas and the example solutions to the assignments.

Runtime comparison

Consider the following functions:

$$T_A(n) = 5n^2 + 3n + 100 \quad (1)$$

$$T_B(n) = 3n^3 - 20n^2 \quad (2)$$

$$T_C(n) = 5n - 10\sqrt{n} + 5 \quad (3)$$

$$T_D(n) = 200 \lg(n + 2) - 10 \quad (4)$$

1. Express each function in Θ notation.
2. For each function, find suitable constants c_1, c_2, n_0 for the Θ form you have found.
3. For each function, find for which $n \geq 1$ each function is positive, and from which n' on it is strictly monotonously increasing.
4. For each function, find the location of its minimum, if it has one.
5. Order the functions in by their asymptotic growth rate. Give a mathematical argument that supports your ordering (compare two functions at a time).
6. Now consider functions in pairs, comparing functions that are "next to each other" in the ordering you found in the last step. For which values of n is the function that is asymptotically larger actually smaller than the asymptotically larger function?

Master theorem

Find a solution to the following recurrence equations using the Master theorem (cases generated by Claude Sonnet 4.6):

$$1. \quad T(n) = 9T\left(\frac{n}{3}\right) + n$$

$$2. \quad T(n) = 8T\left(\frac{n}{2}\right) + n^2$$

$$3. \quad T(n) = T\left(\frac{n}{2}\right) + c$$

$$4. \quad T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

5.
$$T(n) = 3T\left(\frac{n}{4}\right) + n \lg n$$
6.
$$T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\lg n}$$
7.
$$T(n) = 2T\left(\frac{n}{2}\right) + n^2 \sin^2(\lg n) + n$$

Algorithm analysis

Consider the following algorithm, where `A` is an array with one-based indexing. Variable names are intentionally chosen to be uninformative.

```
F(A):
  n = A.length
  q = n
  x = A[q] * A[q]
  for j = n - 1 downto 1
    y = A[j] * A[j]
    if y > x
      q = j
      x = y
  return j
```

- Run it for, e.g., `A = [4, -3, 5, 2, -6]` : What does the algorithm do?
- Give an invariant for the loop.
- Prove that the invariant holds and use it to describe what the algorithm achieves.
- Give a formal definition of the algorithm in terms of input and output.
- Give a runtime analysis of the algorithm.
- The algorithm will fail (trigger an error) in a corner case. What is that corner case and how to fix it?
- Can you suggest better names for algorithm and variables?

Recursive vs iterative solution

- Give an recursive and an iterative algorithm to compute $n!$.
- Provide a runtime analysis of both algorithms!
- Can you give invariants for both algorithms?

Sorting algorithms

Do the sort

Perform each of the sorting algorithms we have studied step-by-step for some input data, e.g., $A = \langle 29, 13, 4, 12, 13, 8 \rangle$. Show each step.

Review invariants and proofs

Review invariants and proofs of correctness for sorting algorithms studied.

Review performance analysis

Review performance analysis for the sorting algorithms studied.

Other simple sorting algorithms?

There is at least one simple sorting algorithm with $O(n^2)$ runtime that we have not explicitly studied in the course. By "simple" I mean that it can be expressed in no more than about 10 lines of pseudocode. Find it, practice it, try to find invariants for all loops and use them to prove correctness. Give a runtime analysis.

Develop algorithms

Unique elements

You are given an array A of integers. Numbers may occur multiple times in A . Can you suggest an algorithm `Unique(A)` that returns an array containing only unique elements? Discuss performance and alternative approaches. Which data structures could be useful for this problem?

Finding the gap

Assume A is an array of n elements containing integers from 1 to $n + 1$ sorted in ascending order. One value is missing. Give an algorithm that finds the missing number. Can you find an algorithm to solve this in $O(\lg n)$ time?

Data Structures

Improved stack and queue

Consider the stack and queue implementations from the lecture. Extend them with a function `size()` that returns the number of elements currently stored on the stack or in the queue. Try to find a solution that is as simple as possible.

Minimum of a linked list

Consider the linked-list implementations from the lecture and Assignment 3. Can you extend the list data structure so that you can find the smallest item stored in the list in constant time?

- How does this affect the runtime for insertion and deletion?
- Consider sorted and unsorted lists.
- Consider singly- and doubly-linked lists.

Graphs

Consider the graph defined by the following adjacency list:

```
Gal = {'a': ['c', 'g', 'h'],  
       'b': ['g', 'h'],  
       'c': ['f', 'c', 'e'],  
       'd': ['b'],  
       'e': ['f', 'g'],  
       'f': ['b', 'c'],  
       'g': ['f'],  
       'h': ['g']}
```

- Draw the graph
- Give the adjacency matrix for the graph
- Perform BFS and DFS for the graph. For BFS, try two different starting nodes.

In []: